

Git & GitHub – Leitfaden für Einsteiger

Praxis-Guide für Teamarbeit, Branch-Management und die wichtigsten Befehle

1. DEIN PROBLEM GELÖST: BRANCH MIT HAUPTBRANCH ÜBERSCHREIBEN

Szenario: Dein lokaler Branch ist nicht mehr synchron mit dem Hauptbranch (`main` / `master`) auf GitHub und du möchtest deinen Branch exakt durch den Stand von GitHub ersetzen.

Schritt 1: `git fetch origin`

Lädt alle aktuellen Daten vom GitHub-Server herunter, ohne deine eigenen Dateien zu verändern.

Schritt 2: `git checkout dein-branch-name`

Wechselt auf den Branch, den du zurücksetzen möchtest.

Schritt 3: `git reset --hard origin/main`

Zwingt deinen Branch auf den exakten Stand von `origin/main`. **Achtung:** Verwirft ungespeicherte lokale Änderungen!

Wichtig bei bereits gepushten Branches:

Falls du deinen alten Branch schon einmal nach GitHub hochgeladen hattest, musst du ihn nach dem Reset einmalig erzwingen nachzuladen:

`git push --force origin dein-branch-name`

2. DER TÄGLICHE WORKFLOW IM TEAM (EMPFEHLUNG)

Regelmäßiges Aktualisieren

Ohne Datenverlust

Bevor du an einem Feature arbeitest, erstelle immer einen neuen Branch vom aktuellen `main`:

- `git checkout main`
- `git pull origin main`
- `git checkout -b mein-neues-feature`

Hauptbranch aktuell halten

Wenn deine Kollegen Änderungen in den `main` gepusht haben, erstelle in deinen Branch die neuesten Updates:

- `git checkout mein-feature`
- `git merge main` (führt Änderungen zusammen)

3. SPICKZETTEL: DIE WICHTIGSTEN GIT-BEFEHLE

BEFEHL	BESCHREIBUNG / EINSATZZWECK
1. START & SETUP	
<code>git clone <url></code>	Lädt ein bestehendes GitHub-Repository auf deinen PC herunter.
<code>git status</code>	Zeigt geänderte, neue oder gelöschte Dateien im aktuellen Branch an.
2. BRANCHING (ABZWEIGE)	
<code>git branch</code>	Zeigt alle lokalen Branches an (der aktuelle ist markiert).
<code>git checkout -b <name></code>	Erstellt einen neuen Branch und wechselt sofort dorthin.
<code>git checkout <name></code>	Wechselt zu einem bestehenden Branch.
3. ÄNDERUNGEN SPEICHERN & HOCHLADEN	
<code>git add .</code>	Bereitet **alle** geänderten Dateien für das Speichern (Commit) vor.
<code>git commit -m "Text"</code>	Speichert den aktuellen Stand lokal mit einer kurzen Beschreibung.
<code>git push origin <name></code>	Lädt deinen lokalen Branch und dessen Commits zu GitHub hoch.
<code>git pull origin main</code>	Holt die neuesten Änderungen von GitHub und fügt sie lokal ein.

4. GOLDENE REGELN FÜR GIT-EINSTEIGER

- **Niemals direkt auf `main` arbeiten:** Erstelle für jede neue Aufgabe einen eigenen Feature-Branch.
- **Oft committen:** Kleine, übersichtliche Commits erleichtern die Fehlersuche enorm.
- **Aussagekräftige Commit-Nachrichten:** Schreibe kurz, was verändert wurde (z.B. `git commit -m "Login-Button hinzugefügt"`).
- **Immer pullen vor der Arbeit:** Starte deinen Arbeitstag immer mit einem `git pull`, um auf dem neuesten Stand zu sein.